TeamChat AI | Technical Brief

Ashraf Al-Madhoun | 8 October 2026 | Phase 1

1. What I will build

I will build a small, usable team chat application with a business assistant that can look up the supplied data, calculate results and remember team decisions. Northside and Valley will use the same application, but each will see only its own stores, messages and memories. The first priority is preventing invented business facts and cross-chain disclosure.

This is my plan before implementation, with delivery on 16 October following Gonzalo's clarification. I will record any design changes and remaining limitations in the README. The scope is a working assessment deployment; production readiness will require further operational testing.

2. User stories and acceptance criteria

Work I will build	How I will check it
As a buyer, I can discuss a decision with my team.	Two users in one room see the same committed message order. Retrying the same client message ID creates one message. Presence expires after a lost connection; typing expires without a stop event.
As a buyer, I can ask @AI for catalog information.	AC, SSD and multi-product questions produce fetched candidates with source references. Reverse lookup works. Missing products return no match. An unspecified product is not silently replaced with a specific one.
As an analyst, I can compare a store's assortment and coefficients.	ST-4003 and ST-4007 use the correct tenant, segment, channel and format. Supersession is applied once per class. Results show inputs, exclusions, formula and any assumed monetary baseline.
As a colleague, I can retrieve a decision from another room.	A paraphrase retrieves a seeded memory with author, original date and source room. A newly saved contact and deadline are retrievable by another user. Invalid nested values receive a useful validation error.
As a user, I can resume a conversation.	A 30-second disconnect replays missed events once in room order. Refreshing during an AI turn restores persisted text and tool records, then follows the same turn.
As a competing chain, I can trust the boundary.	Foreign and nonexistent store IDs give the same outward result. Admin-mode prompts and stored claims of permission cannot change access. Concurrent AI turns have separate IDs and state.

I checked all five fixtures: 1,000 catalog rows, 44 margin classes, 12 segment coefficients, 24 stores and eight memories. There are 289 unmapped catalog entries. Store references resolve to the catalog, and the seeded memory values fit the proposed flat schema.

3. Requirements I derived

Identity alone is insufficient: requests need server-resolved tenant and room membership. A send acknowledgement must mean durable storage. Tool traces must survive refresh, and AI work must survive the browser closing. A saved note is not verified policy or permission. Cross-room memory is deliberately tenant-shared, so the UI must say so before saving. Untrusted text must render as text, not executable HTML. Seeds, migrations and logs need the same boundary checks as normal requests.

Architecture and delivery

4. Technology choices

I will use React and TypeScript on Firebase Hosting, Firebase Auth for identity, and a Node.js/TypeScript API on Cloud Run. Vertex AI Gemini will choose tools through native function calling. Cloud SQL PostgreSQL with pgvector will store business data, messages, events, jobs and memory embeddings generated through Vertex AI. Shared TypeScript contracts keep UI and server interfaces aligned.

The browser will open one native WebSocket to the API. I will not use client database listeners, managed real-time sync, or polling. HTTP is limited to initial loading and commands where appropriate; live room changes arrive through the socket. Tokens go in an initial authentication frame, not a URL. The server checks the Origin allowlist, limits unauthenticated sockets and revalidates membership on commands. Token expiry pauses access until reauthentication; logout closes the connection. Firebase client SDK use is limited to authentication.

Component	Responsibility
React client	Room UI, one connection manager, ordered event reducer, source cards and connection state.
Cloud Run API	Authenticate, authorize, validate commands, commit state, replay events and fan out committed updates.
PostgreSQL	Tenant boundaries, room sequence, deduplication, durable event history, data lookup and memory search.
Cloud Tasks + private Cloud Run worker	Run persisted AI jobs with retries and lease checks. Tasks contain a job ID, not user-supplied authority.
Vertex AI	Select declared tools and produce grounded response structure. It never queries storage directly.

5. How two instances stay in sync

Every durable room change writes an event in the same database transaction as its state change. The transaction also issues PostgreSQL NOTIFY with an opaque room ID and high-water sequence. Each API instance has a dedicated LISTEN connection. On notification, it reads missing authorized events from the database and sends them to its local sockets. The notification is a wake-up signal; the event table is the source of truth.

An instance subscribes before accepting room sockets. The listener stays outside application transactions, with keepalive and a failure deadline. On loss the instance closes affected sockets, reconnects and requires cursor replay. This avoids silently missing events while appearing healthy. Notifications may be duplicated or coalesced; reading from the last sequence handles both. Notifications carry no text or business data; PostgreSQL notifications are not protected by row security. Only trusted server processes see them. Every event fetch and socket recipient is authorized separately. The browser never connects to PostgreSQL.

6. Alternatives and when I would change them

WebSocket over SSE: one channel carries messages, live output and ephemeral presence. SSE plus HTTP is viable, but adds a second interaction path. I would choose it if the product became mostly server-to-client updates.

PostgreSQL over Firestore: relational constraints, transactions, row security and coefficient relationships fit this problem. I would reconsider if document access dominated and the ordering model remained simple.
LISTEN/NOTIFY over Redis: fewer services for this assessment. It costs a dedicated database connection per API instance. At higher connection counts I would move fan-out to Redis or a broker while retaining database replay.

Modular application over many services: one API and one worker deployment, with explicit module boundaries, are enough for a week. Independently scaling modules or different ownership needs would justify further separation. Correctness does not depend on session affinity.

Data and isolation

7. Data model

Records	Keys and purpose
tenants; users; memberships; rooms; room_memberships	Firebase UID maps to server-owned membership. Room access is separate from tenant membership.
messages; room_events	Tenant + room + sequence is unique. Tenant + author + client_message_id deduplicates sends. Events carry event ID, type, turn ID and payload.
ai_jobs; task_outbox; tool_calls; answer_blocks	Job lease, fencing token, attempt, input cutoff and dispatch record. Every tool/output belongs to one tenant, room and turn.
catalog; margin_classes; supersession_edges; segment_coefficients	Tenant-scoped fixture copies. Edges store each supersedes relationship separately. Coefficients use decimal values.
stores; store_carried_codes; store_gaps	Composite tenant/store keys; carried codes and evidence-backed suspected gaps link to catalog records.
memories; memory_versions; presence_leases	Memory keeps author, date, source, flat value and embedding. Per-connection presence leases expire and aggregate across tabs.

Every protected table includes tenant_id. Foreign keys include tenant_id so a record cannot link to another tenant by accident. Catalog and coefficient fixtures are copied into each tenant namespace; there is no unscoped tool query. Indexes support room replay, tenant/store lookup, normalized catalog codes and tenant-filtered memory retrieval. A room sequence is allocated under a row lock inside the write transaction, so committed order is consistent across instances.

8. Where authorization happens

The API verifies Firebase identity, resolves membership through a restricted lookup for that verified UID, and creates a tenant context. Tenant IDs supplied by the browser or Gemini are ignored. Tools receive that context through server code; their declarations do not offer a tenant override. Room subscription, history, replay, job access and tool execution recheck access. Membership changes invalidate existing subscriptions; denied streams close without further delivery. Worker tenant context comes from a restricted internal job lookup, not the task body; that lookup returns only job ownership and is unavailable to public routes or AI tools.

PostgreSQL row-level security provides a second barrier [3]. The runtime role is not a table owner and has no BYPASSRLS privilege; protected tables use forced row security. Each transaction sets a transaction-local tenant context before queries, including worker and vector queries. Connection pooling must never retain one request's tenant for the next. Migrations and seeds use a separate privileged role unavailable to the API.

A foreign store lookup returns the same generic no-match payload as an unknown ID, without naming the chain or confirming existence. Tenant filters run before ranking memory candidates. Cache keys, socket groups and job identifiers are tenant-scoped. Neither an AI instruction nor a saved memory can grant access. An attempted lookup can be logged with the requester, tool and outcome, without fetching or recording a foreign record.

9. What this model makes cheap and expensive

Room replay, exact lookups, coefficient joins and transactional deduplication are straightforward. Repeated embeddings, long transcripts and token-by-token durable writes are costly. I will use bounded answer blocks rather than store every token, index only memory text needed for retrieval, and load history in pages. The per-room lock limits throughput in a very busy room.

Memory values will be flat objects with scalar string, number, boolean or null fields: no arrays or nested objects, at most 20 fields and 8 KB total. Keys and strings have length limits. The schema accepts the supplied memories. A rejected write returns field-level errors for one repair attempt. Lossless flattening is allowed; dropping fields or changing meaning requires user confirmation. Otherwise nothing is saved. Creator and source metadata come from the server, not Gemini.

Real-time and recovery

10. Durable events, then delivery

The client retains a stable message ID until durable acknowledgement. Message, event, AI job and outbox commit together. A duplicate ID with the same payload returns the original result; a different payload is rejected. Delivery is at least once; the client applies each event once by ID and room sequence. Unacknowledged sends survive refresh in a bounded local queue, cleared on logout and never reused by another account.

Before reading a room, the API registers its local subscription and buffers notifications. A repeatable-read transaction captures history, current answer state and event watermark together. New clients receive that snapshot; returning clients replay after their cursor. Events above the watermark then drain in sequence before live delivery. Gaps request replay on the same socket. Batches and buffers are bounded; slow clients disconnect and resume rather than consume unlimited memory.

On network loss the UI shows reconnecting, keeps unacknowledged sends and reconnects with jittered backoff. It retries with the same IDs. Refresh loads persisted history and the current AI job state, then resumes events. Cloud Run can terminate long-lived requests at its configured timeout [1]; reconnect is normal behavior, not an exceptional case.

11. Presence and idle behavior

Presence uses short-lived database leases per authenticated connection; it is not transcript history. Instances batch lease renewals for live sockets and notify peers of renewals, joins and departures. Renewals extend peer expiry timers without resending unchanged presence to browsers. Joining a room reads a fresh authorized presence snapshot. The snapshot excludes expired leases; local expiry timers remove stale entries even after a process dies. Multiple tabs aggregate to one user. Typing is best-effort, room-scoped and expires locally.

Typing expires after five seconds. Leases renew every 20 seconds and expire after 60 seconds; a socket that misses its liveness deadline is no longer renewed. There is one socket and no periodic browser HTTP fetch. A low-frequency WebSocket ping/pong verifies liveness; it is not a data-refresh loop. The idle check will show one connection, with only liveness frames and no repeated data fetches. Request-timeout renewal reconnects and replays normally. A new connection after listener recovery rebuilds both transcript and presence state.

12. AI work survives the socket

The send transaction also writes a task outbox entry. Dispatch creates a Cloud Task with a stable name; an authenticated Cloud Scheduler recovery endpoint retries undispatched entries. This is backend job recovery, never the live transport. Cloud Tasks invokes a private worker using OIDC. The worker holds the HTTP request until completion, with a five-minute turn budget below the configured task deadline. Its renewable database lease and fencing token guard every output write.

Two simultaneous turns have separate context, tool records and output buffers. Each uses the transcript cutoff recorded at submission; it does not accidentally consume a sibling turn's unfinished answer. Tool plans and operation IDs persist before execution. Memory changes and operation results commit atomically after current membership checks. Retries reuse those results. Completed jobs acknowledge without rerunning Gemini; duplicate deliveries of a leased job return a retryable status. Queue and concurrency limits are tenant-scoped; excess requests show a queued state.

Answer blocks are persisted before broadcast. A browser refresh does not cancel the worker. If the worker crashes, a retry rebuilds from recorded tool results and clears or replaces the incomplete answer through explicit revision events; it does not append a new answer to the old partial text. Exhausted retries leave a visible failed state and preserve completed tool traces.

13. Memory and resource limits

Socket close removes subscriptions, timers and listeners. React cleanup removes handlers and aborts stale loads. Server queues, room buffers, database pools and AI context all have limits. Output is batched, history paginated and slow sockets backpressured. Deployment caps API instances and pool sizes against the database connection budget. Repeated connect/disconnect tests will check that socket, listener and timer counts return to baseline; this is evidence to collect during the build, not a claim already proved.

Gemini and business rules

14. Model-driven tools, server-controlled facts

Gemini receives native function declarations for search_catalog, get_catalog_code, get_store_assortment, calculate_uplift, search_memory and save_memory. It chooses tools from the user's language [2]. Detecting the explicit @AI mention starts a turn; it does not choose a business tool. Normalizing separators or expanding AC/SSD/HBA/RTW/DIY inside catalog retrieval improves matching without routing the user's intent by keyword.

The server validates tool name, arguments, tenant context, size and call budget before execution. Results return structured records with immutable source IDs and dataset version. Tool start, arguments, result and failure events are stored in the transcript. A turn has bounded rounds, duration and token usage; malformed calls receive validation feedback rather than unchecked execution.

I will not rely on a prompt telling Gemini to be accurate. The answer contract separates narrative from business facts. Codes, classes, coefficients, store facts and calculations appear in typed source-backed blocks. The server resolves their references against records fetched in that turn and renders the actual values. Unsupported references fail validation. The user-facing renderer accepts only fetched fields, deterministic calculations, cited memory excerpts and fixed transition templates. It does not display arbitrary model-written factual sentences. Clarification blocks use validated missing fields and fetched candidate lists. Validated blocks stream progressively. Raw model tokens containing unchecked facts are never broadcast.

The worker reads Gemini's stream but buffers each incomplete structured block until it can validate and persist it. Users see progressive text/cards and tool activity, rather than unverified token-by-token prose. This is a deliberate UX trade-off. If answer validation fails, the model gets one repair attempt. If it still fails, the user sees verified tool results and an explicit incomplete-answer state. Retrieved memories and tool text are treated as data, including text that claims administrative authority.

15. Coefficients, gaps and missing financial inputs

The files define coefficients and supersession edges but no aggregation formula, sales baseline or calendar for seasons. My explicit calculation assumption is an assortment coefficient score: take distinct mapped margin classes, remove classes superseded by another present class through the transitive relationship, sum the surviving segment-specific margin coefficients, then multiply by the store's channel/format/segment coefficient. Adding a proposed code produces a new score; uplift is the difference. Duplicate classes contribute once. Null mappings are reported as unscored, not assigned a guessed class.

For example, class 318 supersedes 319 and 321, and 319 also supersedes 321. If all three are present, only 318 contributes. The loader checks graph references and rejects cycles. Comparisons show the retained and removed classes so reviewers can inspect the reasoning. This formula is an assessment assumption, not a documented commercial formula; if clarification changes it, I will version the rule and its tests.

There is a fixture conflict: ST-4003 carries APP-100.11, mapped to class 318, and APP-100.99, with no class; its gap evidence says the assortment sits on the unspecified line. I will show both sources and flag the inconsistency. Under the stated score, adding APP-100.12 adds no new class and gives zero uplift. At ST-4007, adding three class-318 codes also contributes class 318 once; class 321 is then superseded. Zero class uplift does not imply zero sales potential.

Gap evidence remains suspected, not confirmed demand. The catalog only says seasonal or year_round, so I will list those flags without inferring summer/winter relevance. A monetary scenario requires a user-supplied baseline and an explicit linear assumption: scenario value = baseline x score difference. Otherwise I return the coefficient difference and ask for the baseline. This is not a sales forecast; score changes are not percentages.

16. Memory retrieval

Memory search uses tenant-filtered embeddings plus lexical matching over the flat value and key. The small fixture set can be ranked exactly within the tenant; there is no need for an approximate index that might hide relevant records. Retrieval returns attribution and original date, not just a generated summary. Weak matches return "I have no record of that." A save acknowledges only after its text and embedding commit. Updates use version checks, retain prior values and expose conflicting decisions rather than silently overwrite them. A retrieved note is attributed as a recorded statement, not confirmed authority. Embedding failure returns a retryable error without pretending the memory is ready.

Ownership, scope and checks

17. How engineers can work independently

Module / owner	Contract and boundary
Client / frontend engineer	Typed commands and event reducers from packages/contracts. Mock socket fixtures allow UI work without a running backend. No database imports.
Identity + data / backend engineer	Verified TenantContext, scoped repositories, migrations and seed loader. Owns access tests and composite keys.
Transport / real-time engineer	Event envelope, replay cursor, acknowledgement and connection lifecycle. Uses repositories; never calls Gemini.
Agent + retail / AI engineer	Tool registry, answer validator, memory retrieval and pure calculation functions. Uses scoped data contracts; never writes to sockets directly.
Delivery / infrastructure engineer	Cloud Run, Cloud SQL, tasks, secrets, deployment and smoke tests. Uses the same repeatable seed and documented environment contract.

I will keep these as modules in one repository, with a separate worker entry point. Shared contracts contain schemas and types, not business implementations. API and worker depend on interfaces for data, model and event output. Module-owned migration files reduce editing conflicts; one integration owner still checks migration order and cross-module constraints. Contract changes require versioned schemas and compatibility checks; integration fixtures cover one complete AI turn. This allows parallel work without introducing several services just to separate ownership.

18. What I am deliberately leaving out

No uploads, rich-text editor, direct messages, mobile app, admin console, SSO, billing or full role hierarchy. Users and rooms are pre-seeded; members can read tenant-shared memory. No live catalog sources, autonomous purchasing or claim that synthetic coefficients predict sales. No multi-region failover or formal production security certification. I will keep the interface clear, with readable source cards and explicit pending/error states, rather than spend the week on animation or branding.

19. Risks and fallback decisions

Grounding: structured answer validation is the hardest boundary. Fixed templates and fetched-value cards keep the factual boundary enforceable; source-reference tests must pass before adding more expressive output. **Replay races:** test notification loss, duplicate delivery and refresh during a turn early. If two-instance delivery fails, I will disclose a single-instance deployment limitation; I will not present it as horizontally verified.

Cloud access and cost: verify Firebase, Vertex AI, Cloud SQL and task permissions first after the brief is submitted. Pin an available Gemini model in configuration and record it. A missing quota or permission is a blocker to report, not a reason to substitute another AI provider. **Calculation ambiguity:** keep the formula explicit, isolated and tested; missing inputs remain missing. **Time:** protect isolation, grounding and recovery before optional UI work.

20. Verification and delivery

Tests will cover foreign/nonexistent lookup equivalence, forged tenant IDs, pooled-connection context, foreign room replay, cross-tenant memory and prompt injection. Calculation tests cover transitive supersession, duplicates, null mappings, the ST-4003 conflict and decimal rounding; grounding tests reject every unfetched reference and unsupported answer field. Transport tests run two instances and two browsers, race snapshot/live events, retry conflicting payloads, interrupt the listener, drop the network and refresh mid-answer. Worker tests will cover task redelivery, lease expiry and idempotent memory writes. A sustained connection test will inspect resource counts and idle traffic.

After submission, I will build identity/seed and isolation, then two-instance chat/replay, then tools and memory. The final day is for verification and the demo. Delivery includes both chains, three users each, genuine AI history, repository, live URL, private test credentials, README, seed, tests and a video under five minutes.

References: [1] Cloud Run WebSockets; [2] Vertex AI function calling; [3] PostgreSQL row security and NOTIFY; [4] Cloud Tasks. Requirements: supplied assessment and seed files; Gonzalo's deadline clarification, 8 October.